

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск набора подстрок в строке

Студент гр. 4345

Гаврилов Е.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм Ахо-Корасик. Применить его для решения задачи нахождения набора подстрок в строке.

Задание

Задание 1

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст

Вторая – число n , каждая из следующих n строк содержит шаблон из набора

Все строки содержат символы из алфавита

Выход:

Все вхождения образцов из S в T .

Каждое вхождение образца в текст представить в виде двух чисел

Где i – позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером j (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номер шаблона.

Вход:

NTAG

3

TAGT

TAG

T

Выход:

2 2

2 3

Задание 2

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с джокером.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу необходимо найти все вхождения в текст.

Например, образец с джокером встречается дважды в тексте xabvссbababсах.

Символ джокер не входит в алфавит, символы которого используются в . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида недопустимы.

Все строки содержат символы из алфавита

Вход:

Текст

Шаблон

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Sample Input:

ACTANCA

A\$\$\$A\$

\$

Sample Output:

1

Задание 3

Вариант 2. Подсчитать количество вершин в автомате; вывести список найденных образцов, имеющих пересечения с другими найденными образцами в строке поиска.

Выполнение работы

В ходе работы был реализован алгоритм Ахо-Корасик, который по входным данным — тексту `text` и набору образцов `patterns` — находит все вхождения образцов набора `patterns` в строке `text` за линейное время (относительно суммарной длины текста и всех образцов) — функция `search(text)` класса `Trie`.

Алгоритм использует бор (`Trie`), составленный из набора образцов, дополненный суффиксными ссылками (поле `suff`) и конечными ссылками (поле `exit`). Суффиксные и конечные ссылки строятся функцией `_build_automaton()` после того, как образцы набора добавлены в бор функцией `_add_word(word, wordIndex)`.

Задание 1

Для решения задания 1 (точный поиск набора образцов) реализована программа, которая считывает из стандартного ввода текст `text`, количество образцов `n` и сами образцы, строит на их основе автомат Ахо-Корасик (класс `Trie`), вызывает на нем `search(text)` для получения списка вхождений, сортирует список по возрастанию позиции вхождения, а при равных позициях — по возрастанию номера образца, и выводит результат.

Описание функции `_add_word(word: str, wordIndex: int) -> None`

На вход подаются строка `word` — добавляемый образец, и его порядковый номер `wordIndex` (нумерация образцов внутри программы начинается с нуля). Обход начинается с текущего узла `current`, равного корню бора `root`. Далее последовательно, по одному символу `char`, происходит проход по строке `word`: если у текущего узла нет дочернего узла для символа `char`, создаётся новый узел `TrieNode` и добавляется в словарь `children` текущего узла; после этого `current` смещается в найденный (или только что созданный) дочерний узел. По окончании обработки всех символов строки `word` текущий узел помечается терминальным (`isTerminal = True`), в него записываются длина добавленного образца (`wordLen`) и его номер (`wordIndex`).

Описание функции `_build_automaton()` -> None

Функция строит суффиксные (suff) и исходящие (exit) ссылки для всех узлов бора, используя обход в ширину (BFS).

Сначала для каждого прямого потомка корня (узла, соответствующего первому символу какого-либо образца) суффиксная ссылка устанавливается непосредственно на корень, и узел добавляется в очередь queue. Далее выполняется цикл, пока очередь не опустеет: из очереди извлекается узел current, и для каждого его дочернего узла child, соответствующего символу char, выполняется следующая обработка:

- в переменную fallback записывается суффиксная ссылка узла current (fallback = current.suff);

- пока fallback не является None и среди детей fallback нет перехода по символу char, значение fallback заменяется на суффиксную ссылку самого fallback (fallback = fallback.suff) — тем самым отыскивается ближайший по суффиксным ссылкам узел, из которого существует переход по символу char;

- если в результате fallback равен None (дошли до корня, у которого нет перехода по char), суффиксная ссылка child устанавливается на корень root; в противном случае она устанавливается на найденный узел fallback.children[char];

- если узел, на который указывает суффиксная ссылка child.suff, является терминальным, исходящая ссылка child.exit устанавливается непосредственно на него; иначе исходящая ссылка child наследуется от исходящей ссылки его суффиксного узла (child.exit = child.suff.exit) — это позволяет впоследствии при поиске проходить по цепочке всех образцов, являющихся суффиксами друг друга и заканчивающихся в одной и той же позиции текста;

- узел child добавляется в очередь queue для последующей обработки его собственных дочерних узлов.

По завершении обхода всех узлов бора автомат Ахо-Корасик считается построенным.

Описание функции `search(text: str) -> List[Tuple[int, int]]`

На вход подаётся строка `text`, в которой производится поиск всех вхождений ранее добавленных образцов. Функция возвращает список пар (позиция начала вхождения, номер образца).

Текущий узел `current` инициализируется корнем бора `root`. Далее производится проход по каждому символу `char` строки `text` с индексом `i`:

- пока текущий узел `current` не является корнем и среди его детей нет перехода по символу `char`, узел `current` заменяется на узел, на который указывает его суффиксная ссылка (`current = current.suff`), — так реализуется переход в автомате при несовпадении символа, аналогично откату в алгоритме Кнута–Морриса–Пратта;

- если после этого среди детей `current` есть переход по `char`, `current` смещается в соответствующий дочерний узел; иначе (переход невозможен даже из корня) `current` остаётся равным корню;

- если получившийся узел `current` терминальный, значит найдено вхождение образца, заканчивающееся в позиции `i`; начальная позиция вхождения вычисляется как `start_pos = i - current.wordLen + 2` (с учётом того, что нумерация позиций в тексте ведётся с единицы), и в список `result` добавляется пара (`start_pos`, `current.wordIndex + 1`);

- далее по цепочке исходящих ссылок (`exit`) организуется поиск всех прочих образцов, заканчивающихся в этой же позиции текста и являющихся суффиксами уже найденного совпадения: пока очередной узел цепочки `temp` не равен `None`, для него аналогично вычисляется начальная позиция вхождения и добавляется соответствующая пара в `result`, после чего `temp` смещается на `temp.exit`.

По окончании прохода по всем символам строки `text` функция возвращает список `result` со всеми найденными вхождениями образцов.

Описание функции `main()`

Из стандартного ввода считываются: строка `text`, целое число `n` — количество образцов, и `n` строк — сами образцы `patterns`. На основе списка

patterns создаётся экземпляр класса Trie, конструктор которого вызывает buildTrie(): все образцы добавляются в бор функцией `_add_word()`, после чего строится автомат функцией `_build_automaton()`. Вызывается `search(text)` для получения списка найденных вхождений results, который сортируется по возрастанию позиции, а при равных позициях — по возрастанию номера образца. В конце концов, стандартный вывод построчно печатаются пары значений (позиция, номер образца) из отсортированного списка.

Задание 2

Для выполнения задания 2 дополнительно реализована функция `buildFromPattern(pattern: str, wildcard: str)` и внесены изменения в `search(text: str)`, `main()`.

Описание функции `buildFromPattern(pattern: str, wildcard: str)`

На вход подаются строка `pattern` — образец с джокерами, и строка `wildcard`, содержащая символ джокера. Образец `pattern` разбивается по вхождениям джокера на список частей `parts` (используется стандартное разбиение строки — `split(wildcard)`). В поле `patternLen` класса `Trie` записывается длина всего образца `pattern` вместе с джокерами, а в поле `wordCount` — количество непустых частей, то есть число сплошных (не разделённых джокером) участков образца, каждый из которых предстоит искать как самостоятельное слово в тексте.

Далее для каждой непустой части `part` из списка `parts` она добавляется в бор функцией `_add_word(part, pos)`, где `pos` — позиция начала данной части внутри исходного образца `pattern`; эта позиция накапливается по мере перебора частей с учётом длины джокера между ними (`pos += len(part) + len(wildcard)`). Позиция `pos` сохраняется в списке `patternStarts` соответствующего терминального узла — список используется, а не одно значение, поскольку одинаковые по содержанию части образца, стоящие на разных позициях, соответствуют одному и тому же узлу бора. После добавления всех частей вызывается `_build_automaton()` для построения автомата, — таким образом,

поиск образца с джокерами сводится к множественному поиску его сплошных частей.

Описание функции `search(text: str) -> List[int]`

Функция построена на основе базового алгоритма Ахо-Корасик: переход по автомату при обработке каждого символа текста выполняется так же, как и в `search()` задания 1 (с откатом по суффиксным ссылкам). Отличие состоит в том, что вместо непосредственного сохранения каждого найденного вхождения части образца как отдельного результата функция подсчитывает потенциальные совпадения полного образца с джокерами в массиве `match_count` размера $\text{len}(\text{text}) + 1$, изначально заполненном нулями.

После перехода в очередной узел `current` для очередного символа текста с индексом i организуется проход по цепочке узлов, начиная с `current` и далее по исходящим ссылкам `node.exit`, пока не будет достигнут корень или `None`. Для каждого терминального узла `node` на этом пути и для каждой сохранённой в нём позиции `part_start` из списка `patternStarts` вычисляется кандидат на позицию начала полного образца в тексте: $\text{start} = i - \text{node.patternLen} - \text{part_start} + 2$. Если полученная позиция `start` попадает в допустимый диапазон $[1, \text{len}(\text{text}) - \text{patternLen} + 1]$ (то есть образец целиком помещается в текст, начиная с этой позиции), значение `match_count[start]` увеличивается на единицу.

После обработки всей строки текста выполняется итоговый проход по всем позициям от 1 до $\text{len}(\text{text})$: если для позиции i накопленное число совпадений `match_count[i]` равно общему числу непустых частей образца `wordCount`, значит все части образца найдены на своих ожидаемых относительных местах, а следовательно, весь образец с джокерами (с учётом любых символов на местах джокеров) начинается в тексте с позиции i . Такая позиция добавляется в список `result`, который возвращается функцией по завершении обработки.

Описание функции `main()`

Из стандартного ввода считываются строка `text`, строка `pattern` (образец с джокером) и строка `wildcard` (символ джокера). Создаётся пустой экземпляр

класса Trie, для которого вызывается `buildFromPattern(pattern, wildcard)`: образец разбивается на части, и по ним строится автомат. Затем вызывается `search(text)`, возвращающая список позиций всех полных вхождений образца, который построчно печатается в стандартный вывод.

Задание 3

Для выполнения задания 3 дополнительно к программе 1 задания реализованы функции `countNodes()` и `findOverlappingMatches(matches)`, а также внесены коррективы в `main()`.

Описание функции `countNodes()` -> int

Функция подсчитывает общее количество вершин построенного автомата. Выполняется обход бора в ширину: в очередь `queue` помещается корень `root`, после чего в цикле, пока очередь не пуста, из неё извлекается очередной узел `node`, счётчик `count` увеличивается на единицу, а все дочерние узлы из `node.children` добавляются в очередь для последующей обработки. По окончании обхода функция возвращает значение `count` — суммарное число всех узлов бора, включая корень.

Описание функции `findOverlappingMatches(matches: List[Tuple[int, int]]) -> List[Tuple[int, int]]`

На вход подаётся список найденных ранее вхождений `matches`, каждый элемент которого — пара `(pos, patternIndex)`: позиция начала вхождения и номер образца. Функция определяет, какие из найденных вхождений пересекаются (перекрываются по занимаемым позициям в тексте) хотя бы с одним другим найденным вхождением.

Для каждого вхождения `(pos, patternIndex)` из `matches` по словарю `wordLens` (заполняемому при добавлении образцов в бор) определяется длина соответствующего образца `wordLen`, и вычисляются границы занимаемого этим вхождением отрезка текста: `start = pos`, `end = pos + wordLen - 1`. Далее для этого вхождения выполняется вложенный проход по всем остальным вхождениям `(other_pos, other_patternIndex)` из `matches` (кроме самого себя): для каждого из них также вычисляются его границы `other_start`, `other_end`, и

проверяется условие пересечения отрезков: $start \leq other_end$ and $other_start \leq end$. Если условие выполнено хотя бы для одного другого вхождения, исходное вхождение (pos , $patternIndex$) признаётся пересекающимся и добавляется в результирующий список `overlapping`.

По завершении обработки всех вхождений возвращается список `overlapping`, содержащий все вхождения, имеющие хотя бы одно пересечение с другими найденными вхождениями.

Описание функции `main()`

Аналогично `main()` задания 1, из стандартного ввода считываются `text`, число образцов `n` и сами образцы, строится автомат (Trie), после чего дополнительно вызывается `countNodes()` для подсчёта и вывода числа вершин автомата. Далее вызывается `search(text)`; полученный список результатов сортируется и выводится построчно. Затем к этому же списку результатов применяется `findOverlappingMatches()`; возвращаемый список также сортируется по тем же правилам (сначала по позиции, затем по номеру образца) и выводится построчно.

Сложность алгоритма

Сложность алгоритма Ахо-Корасик, где M — суммарная длина всех образцов набора `patterns`, N — длина текста `text`, K — число найденных в тексте вхождений образцов:

по времени: $O(M + N + K)$, поскольку за $O(M)$ строится бор и автомат (суффиксные ссылки `suff` и исходящие ссылки `exit` — функции `_add_word` и `_build_automaton`), а далее основным циклом процедуры поиска (функция `search`) текст обрабатывается за $O(N)$, и ещё $O(K)$ уходит на то, чтобы вывести через цепочку `exit`-ссылок каждое дополнительно найденное вхождение; сложением получаем итоговую асимптотику.

по памяти: $O(M + K)$, поскольку нужно создать и хранить бор с автоматом, размер которого зависит от суммарной длины образцов, для которых он создаётся, — $O(M)$. Также создаётся список для хранения

выходных данных — найденных вхождений образцов в текст, который в худшем случае составляет $O(K)$, поэтому оценка худшего случая — $O(M + K)$.

Для задания 2 (поиск образца с джокером), где L — длина образца с джокером (без учёта разбиения на части), N — длина текста:

по времени: $O(L + N)$, поскольку за $O(L)$ строится бор и автомат по непустым частям образца (функция `buildFromPattern`), а поиск по тексту вместе с итоговым проходом по массиву `match_count` (функция `search`) выполняется за $O(N)$.

по памяти: $O(L + N)$, поскольку бор занимает $O(L)$, а массив `match_count` размера $N + 1$ для подсчёта совпадений частей — ещё $O(N)$.

Для задания 3 (подсчёт вершин автомата и пересечений найденных вхождений) — дополнительно к сложности $O(M + N + K)$ поиска:

по времени: ещё $O(M)$ уходит на обход всех вершин бора и подсчёт их числа (функция `countNodes`), и $O(K^2)$ — на попарное сравнение границ всех найденных вхождений между собой (функция `findOverlappingMatches`).

по памяти: $O(K)$ на хранение списка вхождений, имеющих пересечение хотя бы с одним другим найденным вхождением.

Исходный код программы для заданий 1-3 см. в приложении А.

Результаты тестирования функции множественного точного поиска образцов см. в Таблице 1.

Результаты тестирования функции поиска образца с джокером см. в Таблице 2.

Результаты тестирования функции подсчёта вершин автомата и поиска пересекающихся вхождений см. в Таблице 3.

Таблица 1 — Результаты тестирования множественного точного поиска образцов

№	Ввод	Вывод	Комментарий
1	NTAG 3 TAGT TAG T	2 2 2 3	Найден образец TAG и являющийся его частью образец T, начинающийся с той же позиции
2	abrakadabra	1 1	Образец га является

	3 abra ra ka	3 2 8 1 10 2	суффиксом образца abra, оба найжены в разных позициях текста
3	hello world 2 xyz qwe	-	Ни один из образцов не встречается в тексте

Таблица 2 – Результаты тестирования поиска образца с джокером

№	Ввод	Вывод	Комментарий
1	ACTANCA A\$\$\$ \$	1	Единственное вхождение образца с двумя джокерами
2	abababab a\$ab \$	1 6	Образец встречается дважды в разных частях текста
3	xyzxyz q\$\$z \$	-	Фиксированная часть образца (q...z) не встречается в тексте

Таблица 3 – Результаты тестирования подсчёта вершин автомата и пересечений найденных образцов

№	Ввод	Число вершин автомата	Найденные вхождения	Пересекающие ся вхождения	Комментарий
1	NTAG 3 TAGT TAG T	5	2 2 2 3	2 2 2 3	Оба вхождения начинаются с одной позиции и пересекаются друг с другом
2	abrakadabra 3 abra ra ka	9	1 1 3 2 5 3 8 1 10 2	1 1 3 2 8 1 10 2	Образец ka (5 3) не пересекается ни с одним другим найденным вхождением и исключён из результата
3	abcdef 3 ab cd ef	7	1 1 3 2 5 3	-	Образцы идут в тексте подряд без пересечений, ни одно вхождение не пересекается с другим

Вывод

Изучен, реализован алгоритм Ахо-Корасик, и применен для решения задачи множественного поиска образцов в тексте. Реализовано две модифицированных версии алгоритма: для поиска в тексте по образцу с символами-заполнителями (wildcard), и для поиска в тексте с нахождением пересекающихся вхождений паттернов (в соответствии с выданным вариантом 2).

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ
Файл AhoCorasick.py

```
#!/usr/bin/env python3

from collections import deque
from typing import List, Tuple

class TrieNode:
    """Trie node class."""
    def __init__(self) -> None:
        self.isTerminal: bool = False
        self.children: dict[str, TrieNode] = {}
        self.suff: TrieNode | None = None
        self.exit: TrieNode | None = None
        self.wordLen: int | None = None
        self.wordIndex: int | None = None

class Trie:
    """Trie class"""
    def __init__(self, words: List[str] = []) -> None:
        self.root = TrieNode()
        if words:
            self.buildTrie(words)

    def buildTrie(self, words: List[str]) -> None:
        """Build trie with Aho-Corasick automaton from a list of
words"""
        for index, word in enumerate(words):
            self._add_word(word, index)
        self._build_automaton()

    def search(self, text: str) -> List[Tuple[int, int]]:
        """Search for all occurrences of patterns in text using Aho-
Corasick."""
        print("\n" + "="*20)
        print("SEARCHING IN TEXT")
        print(f"Text: '{text}'")
        print("="*20)

        result = []
        current: TrieNode = self.root

        print(f"\nInitial state: current = ROOT (id={id(self.root)})")

        for i, char in enumerate(text):
            print(f"\n--- Processing character '{char}' at position {i}
---")
            print(f"    Current node before transition: id={id(current)}
(isTerminal={current.isTerminal})")

            # follow failure links until we find a match or reach root
            while current is not self.root and char not in
current.children:
```

```

        print(f"    Char '{char}' not in current's children,
following suff link from id={id(current)}")
        current = current.suff
        print(f"        New current: id={id(current)}
(isTerminal={current.isTerminal})")

    # next state
    if char in current.children:
        current = current.children[char]
        print(f"    Char '{char}' found in children, moving to
id={id(current)}")
    else:
        current = self.root
        print(f"    Char '{char}' not found, staying at ROOT
(id={id(self.root)}")

    # check for matches at current node
    if current.isTerminal:
        start_pos = i - current.wordLen + 2
        print(f"        MATCH FOUND at current node!
wordIndex={current.wordIndex}, wordLen={current.wordLen}")
        print(f"        Start position: {start_pos}, Pattern index:
{current.wordIndex + 1}")
        result.append((start_pos, current.wordIndex + 1))

    # check exit links
    temp = current.exit
    if temp is not None:
        print(f"    Checking exit links from id={id(current)}")
        while temp is not None:
            start_pos = i - temp.wordLen + 2
            print(f"        EXIT LINK MATCH! wordIndex={temp.wordIndex},
wordLen={temp.wordLen}")
            print(f"        Start position: {start_pos}, Pattern index:
{temp.wordIndex + 1}")
            result.append((start_pos, temp.wordIndex + 1))
            temp = temp.exit
            if temp is not None:
                print(f"            Following next exit link to
id={id(temp)}")

    print("="*20 + "\n")

    return result

def _add_word(self, word: str, wordIndex: int) -> None:
    """Add one word to Aho-Corasick trie"""
    print(f"Adding word: {word}, word index: {wordIndex}")
    print(f"\nTrie before adding {word}:")
    self._print_trie()

    current = self.root
    for char in word:
        print(f"Adding char: {char} from the word that we are
adding")
        if char not in current.children:
            current.children[char] = TrieNode()
        current = current.children[char]

```

```

        current.isTerminal = True
        current.wordLen = len(word)
        current.wordIndex = wordIndex

        print(f"\nTrie after adding {word}:")
        self._print_trie()

    def _build_automaton(self) -> None:
        """Build the Aho-Corasick automaton by setting up failure links
        (suff)
        and dictionary links (exit) for all nodes."""
        queue = deque()
        print("BUILDING AUTOMATON")
        print("\nTrie before building automaton:")
        self._print_trie()

        for char, child in self.root.children.items():
            # root suff is root
            print(f"Setting suff link for root's child '{char}'
(id={id(child)}) to ROOT (id={id(self.root)})")
            child.suff = self.root
            queue.append(child)

        while queue:
            current = queue.popleft()
            print(f"\nProcessing node id={id(current)}
(isTerminal={current.isTerminal}, wordIndex={current.wordIndex})")

            for char, child in current.children.items():
                print(f" Processing child '{char}' (id={id(child)}) of
node id={id(current)}")

                # start from current suff
                fallback = current.suff
                print(f" Starting fallback from suff link:
id={id(fallback) if fallback else None}")

                # follow suff until we find char as child or reach
                root
                while fallback is not None and char not in
                fallback.children:
                    print(f" Char '{char}' not in fallback's children,
following suff link from id={id(fallback)}")
                    fallback = fallback.suff
                    print(f" New fallback: id={id(fallback) if
                    fallback else None}")

                # set suff
                if fallback is None:
                    child.suff = self.root
                    print(f" Fallback is None, setting suff link to
                    ROOT (id={id(self.root)})")
                else:
                    child.suff = fallback.children[char]
                    print(f" Setting suff link to
                    id={id(child.suff)}")

                # set exit link

```



```

        if child.suff.isTerminal:
            child.exit = child.suff
            print(f"    Suff link node is TERMINAL, setting exit
link to id={id(child.exit)}")
        else:
            child.exit = child.suff.exit
            if child.exit is not None:
                print(f"    Inheriting exit link from suff link:
id={id(child.exit)}")
            else:
                print(f"    No exit link to inherit (None)")

        queue.append(child)
        print(f"    Added child '{char}' (id={id(child)}) to
queue")

    print("\nTrie after building automaton")
    self._print_trie()

def _print_trie(self) -> None:
    print("="*20)
    print("\tTrie nodes (BFS):")

    queue = deque([(self.root, "", "ROOT", id(self.root))])

    while queue:
        node, path, label, wid = queue.popleft()

        desc = f"    {wid}: '{label}' | path='{path}'"
        if node.isTerminal:
            desc += f" | TERMINAL (wordIndex={node.wordIndex},
len={node.wordLen})"
        desc += f" | suff: {id(node.suff)}"
        if node.exit is not None:
            desc += f" | exit: {id(node.exit)}"

        print(desc)

        for char, child in node.children.items():
            queue.append((child, path + char, char, id(child)))

    print("="*20 + "\n")

def main():
    text = input()
    n = int(input())
    words = [input() for _ in range(n)]
    trie = Trie(words)
    results = trie.search(text)
    results.sort(key=lambda x: (x[0], x[1]))
    for position, index in results:
        print(position, index)

if __name__ == "__main__":
    main()

```

Файл WildcardAhoCorasick.py

```

#!/usr/bin/env python3

from collections import deque
from typing import List

class TrieNode:
    """Trie node class."""
    def __init__(self) -> None:
        self.isTerminal: bool = False
        self.children: dict[str, TrieNode] = {}
        self.suff: TrieNode | None = None
        self.exit: TrieNode | None = None
        self.patternLen: int | None = None
        self.patternStarts: List[int] = []

class Trie:
    """Trie class"""
    def __init__(self, words: List[str] = []) -> None:
        self.root = TrieNode()
        self.patternLen: int = 0
        self.wordCount: int = 0
        if words:
            self.buildTrie(words)

    def buildTrie(self, words: List[str]) -> None:
        """Build trie with Aho-Corasick automaton from a list of
words"""
        for index, word in enumerate(words):
            self._add_word(word, index)
            self._build_automaton()

    def buildFromPattern(self, pattern: str, wildcard: str) -> None:
        """Split a wildcard pattern into fixed parts and build the
automaton
needed to find all full matches of the pattern in a text."""
        parts = pattern.split(wildcard)

        self.patternLen = len(pattern)
        self.wordCount = sum(1 for part in parts if part)

        pos = 0
        for part in parts:
            if part:
                self._add_word(part, pos)
                pos += len(part) + len(wildcard)

        self._build_automaton()

    def search(self, text: str) -> List[int]:
        """Search for all occurrences of the wildcard pattern in text
using Aho-Corasick."""
        print("\n" + "="*20)
        print("SEARCHING IN TEXT")
        print(f"Text: '{text}'")
        print("="*20)

        match_count = [0] * (len(text) + 1)
        current: TrieNode = self.root

```

```

print(f"\nInitial state: current = ROOT (id={id(self.root)})")

for i, char in enumerate(text):
    print(f"\n--- Processing character '{char}' at position {i}
---")
    print(f"    Current node before transition: id={id(current)}
(isTerminal={current.isTerminal})")

    # follow failure links until we find a match or reach root
    while current is not self.root and char not in
current.children:
        print(f"    Char '{char}' not in current's children,
following suff link from id={id(current)}")
        current = current.suff
        print(f"        New current: id={id(current)}
(isTerminal={current.isTerminal})")

    # next state
    if char in current.children:
        current = current.children[char]
        print(f"    Char '{char}' found in children, moving to
id={id(current)}")
    else:
        current = self.root
        print(f"    Char '{char}' not found, staying at ROOT
(id={id(self.root)})")

    # check for matches at current node and along exit links
    node: TrieNode | None = current
    while node is not None and node is not self.root:
        if node.isTerminal:
            print(f"    MATCH FOUND at node id={id(node)}!
patternLen={node.patternLen}, starts={node.patternStarts}")
            for part_start in node.patternStarts:
                start = i - node.patternLen - part_start + 2
                print(f"        Candidate pattern start position:
{start}")
                if 1 <= start <= len(text) - self.patternLen +
1:
                    match_count[start] += 1
                    print(f"        -> counted toward pattern start
{start} (count={match_count[start]})")
                    node = node.exit
                    if node is not None and node is not self.root:
                        print(f"        Following exit link to id={id(node)}")

print("\nChecking accumulated match counts against required word
count ")
    f"({self.wordCount}):")
    result = []
    for i in range(1, len(text) + 1):
        print(f"    Position {i}: match_count={match_count[i]}")
        if match_count[i] == self.wordCount:
            print(f"        -> Position {i} matches full pattern")
            result.append(i)

print("=*20 + "\n")

```

```

        return result

    def _add_word(self, word: str, wordStart: int) -> None:
        """Add one word to Aho-Corasick trie"""
        print(f"Adding word: {word}, word start: {wordStart}")
        print(f"\nTrie before adding {word}:")
        self._print_trie()

        current = self.root
        for char in word:
            print(f"Adding char: {char} from the word that we are
adding")
            if char not in current.children:
                current.children[char] = TrieNode()
                current = current.children[char]
            current.isTerminal = True
            current.patternLen = len(word)
            current.patternStarts.append(wordStart)

        print(f"\nTrie after adding {word}:")
        self._print_trie()

    def _build_automaton(self) -> None:
        """Build the Aho-Corasick automaton by setting up failure links
(suff)
and dictionary links (exit) for all nodes."""
        queue = deque()
        print("BUILDING AUTOMATON")
        print("\nTrie before building automaton:")
        self._print_trie()

        for char, child in self.root.children.items():
            # root suff is root
            print(f"Setting suff link for root's child '{char}'
(id={id(child)}) to ROOT (id={id(self.root)})")
            child.suff = self.root
            queue.append(child)

        while queue:
            current = queue.popleft()
            print(f"\nProcessing node id={id(current)}
(isTerminal={current.isTerminal},
patternStarts={current.patternStarts})")

            for char, child in current.children.items():
                print(f" Processing child '{char}' (id={id(child)}) of
node id={id(current)}")

                # start from current suff
                fallback = current.suff
                print(f" Starting fallback from suff link:
id={id(fallback) if fallback else None}")

                # follow suff until we find char as child or reach
root
                while fallback is not None and char not in
fallback.children:

```

```

        print(f"    Char '{char}' not in fallback's children,
following suff link from id={id(fallback)}")
        fallback = fallback.suff
        print(f"    New fallback: id={id(fallback) if
fallback else None}")

    # set suff
    if fallback is None:
        child.suff = self.root
        print(f"    Fallback is None, setting suff link to
ROOT (id={id(self.root)}))")
    else:
        child.suff = fallback.children[char]
        print(f"    Setting suff link to
id={id(child.suff)}")

    # set exit link
    if child.suff.isTerminal:
        child.exit = child.suff
        print(f"    Suff link node is TERMINAL, setting exit
link to id={id(child.exit)}")
    else:
        child.exit = child.suff.exit
        if child.exit is not None:
            print(f"    Inheriting exit link from suff link:
id={id(child.exit)}")
        else:
            print(f"    No exit link to inherit (None)")

    queue.append(child)
    print(f"    Added child '{char}' (id={id(child)}) to
queue")

    print("\nTrie after building automaton")
    self._print_trie()

    def _print_trie(self) -> None:
        print("="*20)
        print("\tTrie nodes (BFS):")

        queue = deque([(self.root, "", "ROOT", id(self.root))])

        while queue:
            node, path, label, wid = queue.popleft()

            desc = f"    {wid}: '{label}' | path='{path}'"
            if node.isTerminal:
                desc += f"    |    TERMINAL"
            (patternStarts={node.patternStarts}, len={node.patternLen})"
            desc += f"    | suff: {id(node.suff)}"
            if node.exit is not None:
                desc += f"    | exit: {id(node.exit)}"

            print(desc)

            for char, child in node.children.items():
                queue.append((child, path + char, char, id(child)))

```

```

        print(""*20 + "\n")

def main():
    text = input()
    pattern = input()
    wildcard = input()

    trie = Trie()
    trie.buildFromPattern(pattern, wildcard)

    results = trie.search(text)
    for position in results:
        print(position)

if __name__ == "__main__":
    main()

```

Файл OverlapAhoCorasick.py

```

#!/usr/bin/env python3

from collections import deque
from typing import List, Tuple

class TrieNode:
    """Trie node class."""
    def __init__(self) -> None:
        self.isTerminal: bool = False
        self.children: dict[str, TrieNode] = {}
        self.suff: TrieNode | None = None
        self.exit: TrieNode | None = None
        self.wordLen: int | None = None
        self.wordIndex: int | None = None

class Trie:
    """Trie class"""
    def __init__(self, words: List[str] = []) -> None:
        self.root = TrieNode()
        self.wordLens: dict[int, int] = {}
        if words:
            self.buildTrie(words)

    def buildTrie(self, words: List[str]) -> None:
        """Build trie with Aho-Corasick automaton from a list of
words"""
        for index, word in enumerate(words):
            self._add_word(word, index)
            self._build_automaton()

    def countNodes(self) -> int:
        """Count the total number of vertices (nodes) in the
automaton."""
        print("\n" + "*"20)
        print("COUNTING AUTOMATON VERTICES")
        print(""*20)

        count = 0
        queue = deque([self.root])

```

```

while queue:
    node = queue.popleft()
    count += 1
    print(f"    Counting node id={id(node)} (total so far:
{count}))")

    for char, child in node.children.items():
        queue.append(child)

print(f"Total number of vertices: {count}")
print("="*20 + "\n")

return count

def findOverlappingMatches(self, matches: List[Tuple[int, int]]) ->
List[Tuple[int, int]]:
    print("\n" + "="*20)
    print("CHECKING FOUND PATTERNS FOR INTERSECTIONS")
    print(f"Found matches: {matches}")
    print("="*20)

    overlapping: List[Tuple[int, int]] = []

    for idx, (pos, patternIndex) in enumerate(matches):
        wordLen = self.wordLens[patternIndex - 1]
        start = pos
        end = pos + wordLen - 1
        print(f"\n    Checking pattern {patternIndex} at position
{pos} (occupies [{start}, {end}])")

        has_overlap = False
        for other_idx, (other_pos, other_patternIndex) in
enumerate(matches):
            if other_idx == idx:
                continue
            other_len = self.wordLens[other_patternIndex - 1]
            other_start = other_pos
            other_end = other_pos + other_len - 1

            if start <= other_end and other_start <= end:
                print(f"                Overlaps with pattern
{other_patternIndex} at position {other_pos} "
                    f"(occupies [{other_start}, {other_end}])")
                has_overlap = True

        if has_overlap:
            print(f"    -> Pattern {patternIndex} at position {pos}
has at least one intersection")
            overlapping.append((pos, patternIndex))
        else:
            print(f"    -> Pattern {patternIndex} at position {pos}
has no intersections")

    print(f"\nMatches with intersections: {overlapping}")
    print("="*20 + "\n")

    return overlapping

```

```

def search(self, text: str) -> List[Tuple[int, int]]:
    """Search for all occurrences of patterns in text using Aho-
Corasick."""
    print("\n" + "="*20)
    print("SEARCHING IN TEXT")
    print(f"Text: '{text}'")
    print("="*20)

    result = []
    current: TrieNode = self.root

    print(f"\nInitial state: current = ROOT (id={id(self.root)})")

    for i, char in enumerate(text):
        print(f"\n--- Processing character '{char}' at position {i}
---")
        print(f"    Current node before transition: id={id(current)}
(isTerminal={current.isTerminal})")

        # follow failure links until we find a match or reach root
        while current is not self.root and char not in
current.children:
            print(f"    Char '{char}' not in current's children,
following suff link from id={id(current)}")
            current = current.suff
            print(f"        New current: id={id(current)}
(isTerminal={current.isTerminal})")

            # next state
            if char in current.children:
                current = current.children[char]
                print(f"    Char '{char}' found in children, moving to
id={id(current)}")
            else:
                current = self.root
                print(f"    Char '{char}' not found, staying at ROOT
(id={id(self.root)})")

            # check for matches at current node
            if current.isTerminal:
                start_pos = i - current.wordLen + 2
                print(f"        MATCH FOUND at current node!
wordIndex={current.wordIndex}, wordLen={current.wordLen}")
                print(f"        Start position: {start_pos}, Pattern index:
{current.wordIndex + 1}")
                result.append((start_pos, current.wordIndex + 1))

            # check exit links
            temp = current.exit
            if temp is not None:
                print(f"    Checking exit links from id={id(current)}")
                while temp is not None:
                    start_pos = i - temp.wordLen + 2
                    print(f"    EXIT LINK MATCH! wordIndex={temp.wordIndex},
wordLen={temp.wordLen}")
                    print(f"        Start position: {start_pos}, Pattern index:
{temp.wordIndex + 1}")

```



```

        result.append((start_pos, temp.wordIndex + 1))
        temp = temp.exit
        if temp is not None:
            print(f"          Following next exit link to
id={id(temp)}")

        print(" "*20 + "\n")

        return result

def _add_word(self, word: str, wordIndex: int) -> None:
    """Add one word to Aho-Corasick trie"""
    print(f"Adding word: {word}, word index: {wordIndex}")
    print(f"\nTrie before adding {word}:")
    self._print_trie()

    current = self.root
    for char in word:
        print(f"Adding char: {char} from the word that we are
adding")
        if char not in current.children:
            current.children[char] = TrieNode()
            current = current.children[char]
        current.isTerminal = True
        current.wordLen = len(word)
        current.wordIndex = wordIndex
        self.wordLens[wordIndex] = len(word)

    print(f"\nTrie after adding {word}:")
    self._print_trie()

def _build_automaton(self) -> None:
    """Build the Aho-Corasick automaton by setting up failure links
(suff)
and dictionary links (exit) for all nodes."""
    queue = deque()
    print("BUILDING AUTOMATON")
    print("\nTrie before building automaton:")
    self._print_trie()

    for char, child in self.root.children.items():
        # root suff is root
        print(f"Setting suff link for root's child '{char}'
(id={id(child)}) to ROOT (id={id(self.root)})")
        child.suff = self.root
        queue.append(child)

    while queue:
        current = queue.popleft()
        print(f"\nProcessing node id={id(current)}
(isTerminal={current.isTerminal}, wordIndex={current.wordIndex})")

        for char, child in current.children.items():
            print(f"  Processing child '{char}' (id={id(child)}) of
node id={id(current)}")

            # start from current suff
            fallback = current.suff

```

```

        print(f"        Starting fallback from suff link:
id={id(fallback) if fallback else None}")

        # follow suff until we find char as child or reach
root
        while fallback is not None and char not in
fallback.children:
            print(f"        Char '{char}' not in fallback's children,
following suff link from id={id(fallback)}")
            fallback = fallback.suff
            print(f"        New fallback: id={id(fallback) if
fallback else None}")

        # set suff
        if fallback is None:
            child.suff = self.root
            print(f"        Fallback is None, setting suff link to
ROOT (id={id(self.root)}")
        else:
            child.suff = fallback.children[char]
            print(f"        Setting suff link to
id={id(child.suff)}")

        # set exit link
        if child.suff.isTerminal:
            child.exit = child.suff
            print(f"        Suff link node is TERMINAL, setting exit
link to id={id(child.exit)}")
        else:
            child.exit = child.suff.exit
            if child.exit is not None:
                print(f"        Inheriting exit link from suff link:
id={id(child.exit)}")
            else:
                print(f"        No exit link to inherit (None)")

        queue.append(child)
        print(f"        Added child '{char}' (id={id(child)}) to
queue")

    print("\nTrie after building automaton")
    self._print_trie()

    def _print_trie(self) -> None:
        print("="*20)
        print("\tTrie nodes (BFS):")

        queue = deque([(self.root, "", "ROOT", id(self.root))])

        while queue:
            node, path, label, wid = queue.popleft()

            desc = f"    {wid}: '{label}' | path='{path}'"
            if node.isTerminal:
                desc += f" | TERMINAL (wordIndex={node.wordIndex},
len={node.wordLen})"
            desc += f" | suff: {id(node.suff)}"
            if node.exit is not None:

```

```

        desc += f" | exit: {id(node.exit)}"

    print(desc)

    for char, child in node.children.items():
        queue.append((child, path + char, char, id(child)))

    print("="*20 + "\n")

def main():
    text = input()
    n = int(input())
    words = [input() for _ in range(n)]
    trie = Trie(words)

    node_count = trie.countNodes()
    print(f"Number of automaton vertices: {node_count}")

    results = trie.search(text)
    results.sort(key=lambda x: (x[0], x[1]))
    for position, index in results:
        print(position, index)

    overlapping = trie.findOverlappingMatches(results)
    overlapping.sort(key=lambda x: (x[0], x[1]))
    print("Found patterns intersecting with other found patterns:")
    for position, index in overlapping:
        print(position, index)

if __name__ == "__main__":
    main()

```